

Refine — IAM Permissions Reference

Purpose: AWS Marketplace reviewers and customer security teams need to know exactly what IAM permissions the Refine CloudFormation stack creates in the customer's account. This document is the authoritative reference.

Refine is **read-only by design**. Across both the host stack (Marketplace-deployed) and the per-account onboarding stack (added once per AWS account being monitored), Refine never requests, accepts, or uses write permissions on customer cloud resources.

Stack A — Marketplace Host Stack (`refine-marketplace-cfn.yaml`)

This is the stack the customer launches from the AWS Marketplace listing. It provisions the Refine EC2 host in the customer's account.

IAM resources created

Resource	Type	Purpose
<code>RefineEc2Role</code>	<code>AWS::IAM::Role</code>	Attached to the Refine EC2 instance. The minimum to verify the Marketplace entitlement through AWS License Manager, run SSM Session Manager, store the generated admin password in the customer's own SSM Parameter Store, read the delivery bucket (direct-sales / GovCloud path) and assume the per-account read roles that Stack B creates.
<code>RefineInstanceProfile</code>	<code>AWS::IAM::InstanceProfile</code>	Wraps the role for EC2 attachment.

Both are created only when `CreateInstanceProfile=true` (the default). The customer may bring their own instance profile via `ExistingInstanceProfileArn`, in which case the stack creates **no** IAM resources.

Policies attached to `RefineEc2Role`

AWS-managed: `AmazonSSMManagedInstanceCore` (SSM Session Manager — browser-based shell, no SSH key or inbound port).

Inline, in the order the template declares them:

Policy	Actions	Resource
<code>DeliveryBucketRead</code> — BYOL only (the policy is absent from Contract and Free hosts)	<code>s3:GetObject</code>	<code><delivery-bucket>/refine.license</code> (the signed licence the direct-sales watcher pulls)
	<code>s3:ListBucket</code>	<code><delivery-bucket></code>
<code>MarketplaceLicenseManager</code>	<code>license-manager:CheckoutLicense</code> , <code>license-manager:GetLicense</code> , <code>license-manager:CheckInLicense</code> , <code>license-</code> <code>manager:ExtendLicenseConsumption</code> , <code>license-</code> <code>manager:ListReceivedLicenses</code>	* — these APIs are not resource-scoped; access is gated by the product and the buyer's contract
	<code>iam:CreateServiceLinkedRole</code>	<code>role/aws-service-role/license-</code> <code>manager.amazonaws.com/*</code> , condition <code>iam:AWSServiceName = license-</code> <code>manager.amazonaws.com</code> (first entitlement check in an account that never used License Manager)
<code>RefineAdminSecret</code>	<code>ssm:PutParameter</code> , <code>ssm:AddTagsToResource</code>	<code>parameter/refine/<StackName>/*</code> — the generated first-login admin password, written once as a <code>SecureString</code> the customer reads back from their own account
<code>AssumeMonitoredAccountRoles</code>	<code>sts:AssumeRole</code>	<code>arn:</code> <code><partition>:iam::*:role/blacktip_refine-</code> <code>role-*</code> — the External-ID-protected read roles Stack B creates in each monitored account; nothing is assumable until the customer launches that stack

The GovCloud host template (`refine-marketplace-cfn-govcloud.yaml`) carries the same role **without** the `MarketplaceLicenseManager` and `RefineAdminSecret` policies (no Marketplace entitlement there; the licence is delivered as a signed file).

Non-IAM resources the stack creates (for completeness)

- 1× EC2 instance (default `t3.medium` , customer-sizable up to `m6i.4xlarge`).
- 1× EBS root volume (default 30 GB gp3).
- 1× S3 bucket (`refine-delivery-<stack>-<account>` , `DeletionPolicy: Retain`) — **BYOL / direct-sales only** (`CreateDeliveryBucket` = new bucket **and** BYOL mode): it receives the signed licence and survives stack deletion. Contract and Free hosts create no bucket and carry no S3 grant (before v3.14.8 they got an empty retained bucket).
- 1× Security Group (inbound: SSH from `AllowedSshCidr` , app port 8000 from `AllowedAppCidr`).
- Optional: 1× Elastic IP.
- Optional: 1× Application Load Balancer + ACM listener + target group + ALB Security Group (when `EnableHttps=true`).

No Lambda, no IAM users, no IAM access keys, no Secrets Manager secrets.

Stack B — Per-Account Onboarding Stack (`customer-onboarding-v2.yaml`)

This stack is launched once **per AWS account the customer wants Refine to monitor**. It's separate from the Marketplace stack and gets launched from inside the running Refine UI (Cloud Accounts → Add Account → click "Launch CloudFormation"). Marketplace doesn't ingest this template directly, but reviewers commonly ask about it. The exact statements are in `docs/operations/AWS_IAM_PERMISSIONS.md`; `tests/test_iam_docs.py` checks both documents against the template on every build.

IAM resources created

Resource	Type	Purpose
<code>BlacktipRefineRole</code> (<code>blacktip_refine-role-<StackName></code>)	<code>AWS::IAM::Role</code>	Assumed by the Refine host's instance role (cross-account, External-ID-protected). Reads the summary bucket and invokes the collector.
<code>LambdaExecutionRole</code> (<code>blacktip_refine-lambda-role-<StackName></code>)	<code>AWS::IAM::Role</code>	Execution role of the per-account collector Lambda: read-only across the services Refine analyzes, plus write to the customer-owned summary bucket.
<code>LogGroupCleanupRole</code> (<code>blacktip_refine-log-cleanup-<StackName></code>)	<code>AWS::IAM::Role</code>	Execution role of a custom-resource Lambda that tags the stack's own log groups and deletes them on stack deletion.

| `RefineServiceUser` (`refine-service-<StackName>`) + `RefineServiceAccessKey` | `AWS::IAM::User` , `AWS::IAM::AccessKey` | **Only** when the stack is launched without `RefineHostRoleArn` (access-key mode for direct-sales hosts that have no IAM role). Its single inline policy allows `sts:AssumeRole` on `BlacktipRefineRole` and nothing else. The Marketplace host passes its instance role ARN, so on the Marketplace path no user and no key exist. |

No managed policies are created.

What `BlacktipRefineRole` (the role Refine assumes) can do

Actions	Resource
<code>s3:GetObject</code> , <code>s3:ListBucket</code> , <code>s3:GetBucketLocation</code>	the stack's summary bucket only
<code>lambda:InvokeFunction</code>	the stack's collector function only (Sync Now)
<code>lambda:GetFunctionConfiguration</code>	<code>function:blacktip_refine-data-collector-*</code> (reads the template version)

It cannot describe, list or read a single customer resource: everything Refine shows comes from the summary files the collector wrote to the customer's bucket.

What `LambdaExecutionRole` (the collector) can do

Read-only, one statement per service, each action named explicitly (no `Describe*` wildcards except `savingsplans:Describe*` / `savingsplans:List*`):

Category	Actions
Compute	<code>ec2:DescribeInstances</code> , <code>ec2:DescribeVolumes</code> , <code>ec2:DescribeSnapshots</code> , <code>ec2:DescribeNatGateways</code> , <code>ec2:DescribeSecurityGroups</code> , <code>ec2:DescribeVpcs</code> , <code>ec2:DescribeSubnets</code> , <code>ec2:DescribeRegions</code> , <code>ec2:DescribeTags</code>
Database	<code>rds:DescribeDBInstances</code> , <code>rds:DescribeDBClusters</code> , <code>rds:DescribeReservedDBInstances</code> , <code>rds:ListTagsForResource</code>
Cost (commercial template only)	<code>ce:GetCostAndUsage</code> , <code>ce:GetCostAndUsageWithResources</code> , <code>ce:GetCostForecast</code> , <code>ce:GetDimensionValues</code> , <code>ce:GetTags</code> , <code>ce:GetSavingsPlansUtilization</code> , <code>ce:GetReservationUtilization</code> , <code>ce:GetSavingsPlansCoverage</code>
Commitments	<code>savingsplans:Describe*</code> , <code>savingsplans:List*</code>
Monitoring	<code>cloudwatch:GetMetricStatistics</code> , <code>cloudwatch:GetMetricData</code> , <code>cloudwatch:ListMetrics</code> , <code>logs:DescribeLogGroups</code> , <code>logs:DescribeLogStreams</code> , <code>logs:ListTagsLogGroup</code>
Storage	<code>s3:ListAllMyBuckets</code> , <code>s3:GetBucketLocation</code> , <code>s3:GetLifecycleConfiguration</code> , <code>s3:GetBucketVersioning</code> , <code>s3:GetBucketTagging</code> , <code>s3:GetEncryptionConfiguration</code> , <code>s3:GetIntelligentTieringConfiguration</code> , <code>s3:ListBucket</code>
Network	<code>elasticloadbalancing:DescribeLoadBalancers</code> , <code>elasticloadbalancing:DescribeTargetGroups</code> , <code>elasticloadbalancing:DescribeTargetHealth</code> , <code>elasticloadbalancing:DescribeTags</code>
Serverless	<code>lambda:ListFunctions</code> , <code>lambda:GetFunctionConfiguration</code> , <code>lambda:ListProvisionedConcurrencyConfigs</code> , <code>lambda:ListTags</code>
Containers	<code>ecs:ListClusters</code> , <code>ecs:ListServices</code> , <code>ecs:DescribeServices</code> , <code>ecs:DescribeTaskDefinition</code> , <code>ecs:ListTagsForResource</code>
Cache	<code>elasticache:DescribeReplicationGroups</code> , <code>elasticache:DescribeCacheClusters</code> , <code>elasticache:ListTagsForResource</code>
Batch, Lightsail	<code>batch:DescribeComputeEnvironments</code> , <code>batch:DescribeJobQueues</code> , <code>batch:ListJobs</code> , <code>batch:DescribeJobs</code> , <code>lightsail:GetInstances</code> , <code>lightsail:GetInstance</code> , <code>lightsail:GetInstanceMetricData</code>
Write — the single exception	<code>s3:PutObject</code> , <code>s3:GetObject</code> , <code>s3:ListBucket</code> scoped to the stack's own summary bucket
E-mail (only when the <code>ReportEmail</code> parameter is set)	<code>ses:SendEmail</code> on <code>identity/<ReportEmail></code> — the monthly savings e-mail from the customer's own SES identity

Plus the AWS-managed `AWSLambdaBasicExecutionRole` (its own CloudWatch logs). The Lambda makes no network calls outside AWS and uses no third-party SDKs.

What `LogGroupCleanupRole` can do

`logs>DeleteLogGroup`, `logs:TagLogGroup`, `logs:TagResource`, `logs:ListTagsLogGroup` on `log-group:/aws/lambda/blacktip_refine-*` in the account, and `logs:DescribeLogGroups` (not resource-scoped), plus `AWSLambdaBasicExecutionRole`. It exists so that deleting the stack removes the collector's log groups; it touches nothing else.

Trust policy

`BlacktipRefineRole` can only be assumed, and only with a matching `sts:ExternalId` (random per customer; shown in the Refine UI), by:

- the Refine host's instance role (`RefineHostRoleArn` , keyless mode — the Marketplace path), or
- the stack's own `RefineServiceUser` (access-key mode), and
- the stack's own collector execution role (so a self-hosted install can sync locally).

No principal outside the customer's account is trusted; Blacktip Solutions cannot assume the role.

This is the standard third-party cross-account access pattern documented at https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_create_for-user_externalid.html.

Stack C — GovCloud Onboarding Stack (`customer-onboarding-govcloud.yaml`)

Identical to Stack B with two differences:

- Partition switched to `aws-us-gov` throughout the IAM resource ARNs.
- The Cost Explorer statement and the SES e-mail statement are **dropped** (Cost Explorer is not available in AWS GovCloud; commitment figures come from the linked commercial account's stack).

Verification commands (for the customer's security team)

After Stack A is deployed, the customer can verify the host's permissions:

```
# What the EC2 instance role can do – should be only the policies listed above.
ROLE=$(aws cloudformation describe-stack-resource --stack-name <host-stack> \
  --logical-resource-id RefineEc2Role --query StackResourceDetail.PhysicalResourceId --output text)
aws iam list-attached-role-policies --role-name "$ROLE"
aws iam list-role-policies --role-name "$ROLE"
aws iam get-role-policy --role-name "$ROLE" --policy-name MarketplaceLicenseManager
```

After Stack B is deployed in a monitored account:

```
# Confirm the roles are read-only – the only write actions are s3:PutObject on the summary
# bucket (collector) and the log-group tagging/deletion of the stack's own log groups.
aws iam get-role --role-name blacktip_refine-role-<StackName>
aws iam list-role-policies --role-name blacktip_refine-role-<StackName>
aws iam get-role-policy --role-name blacktip_refine-lambda-role-<StackName> --policy-name DataCollectionPolicy
```

For an automated audit, IAM Access Analyzer should report **zero external access risk** on either role beyond the explicitly intended cross-account trust to the Refine host.

Why this matters for AWS Marketplace review

- Refine ships **zero write permissions** on customer resources outside of writing summary files to a bucket the customer owns. This is the foundational claim of the product.

- The Marketplace host stack creates **no IAM permissions for the customer's monitored cloud accounts**. Customers explicitly opt-in by launching Stack B in each account.
- Cross-account access uses **the AWS-recommended ExternalID pattern** with rotating IDs per customer.
- All customer cost/inventory data lives **in the customer's own S3 / Blob / GCS bucket** — Refine reads from there. Blacktip Solutions has no network path to customer data.

© 2026 Blacktip Solutions · Indian Land, SC · WOSB · VOSB · CAGE 14QN0 · UEI SU55FSWCWK98

© 2026 Blacktip Solutions · Indian Land, SC · Corporate@blacktip-ops.com
WOSB · VOSB · CAGE 14QN0 · UEI SU55FSWCWK98